

CHAPTER 2 Number Systems

[Numeral system \(wikipedia\)](#)

“ It’s not about the number of hours you practice, it’s about the number of hours your mind is present during the practice.”

— Kobe Bryant



0 1 2 3 4 5 6 7 8 9
· ۰ ١ ٢ ٣ ٤ ٥ ٦ ٧ ٨ ٩
I II III IV V VI VII VIII IX X
௦ ௧ ௨ ௩ ௪ ௫ ௬ ௭ ௮ ௯
൦ ൧ ൨ ൩ ൪ ൫ ൬ ൭ ൮ ൯
௦ ௧ ௨ ௩ ௪ ௫ ௬ ௭ ௮ ௯
〇 一 二 三 四 五 六 七 八 九

Objectives

- Understand the concept of number systems.
- Distinguish between non-positional and positional number systems.
- Describe the decimal, binary, hexadecimal, and octal system.
- Convert a number in binary, octal, or hexadecimal to a number in the decimal system.
- Convert a number in the decimal system to a number in binary, octal, and hexadecimal.
- Convert a number in binary to octal and vice versa.
- Convert a number in binary to hexadecimal and vice versa.
- Find the number of digits needed in each system to represent a particular value.

2.1 INTRODUCTION

- A number system defines how a number can be represented using distinct symbols.
- A number can be represented differently in different systems.
 - $42 = (42)_{10} = (101010)_2 = (2A)_{16} = (52)_8$
 - their representations are different.
- Several number systems have been used in the past and can be categorized into two groups:
 - **Positional number system**
 - **Non-positional number system**
 - Our main goal is to discuss the positional number systems, but we also give examples of non-positional systems.

2.2 POSITIONAL NUMBER SYSTEMS

- In a positional number system, the position a symbol occupies in the number determines the value it represents.
- In this system, a number represented as:

$\pm (s_{k-1}s_{k-2} \dots s_1s_0.s_{-1}s_{-2} \dots s_{-l})_b$ has the value of:

$n = \pm$	s_{k-1}	s_{k-2}	$\dots$	s_1	s_0	$.$	s_{-1}	s_{-2}	$\dots$	s_{-l}			
	$\times$	$+$	$\times$	$+$	$\dots$	$+$	$\times$	$+$	$\times$	$+$	$\dots$	$+$	$\times$
	b^{k-1}	b^{k-2}	b^{k-3}	b^1	b^0		b^{-1}	b^{-2}	b^{-3}	b^{-l}			

$$n = \pm s_{k-1} \times b^{k-1} + s_{k-2} \times b^{k-2} + \dots + s_1 \times b^1 + s_0 \times b^0 + s_{-1} \times b^{-1} + s_{-2} \times b^{-2} + \dots + s_{-l} \times b^{-l}$$

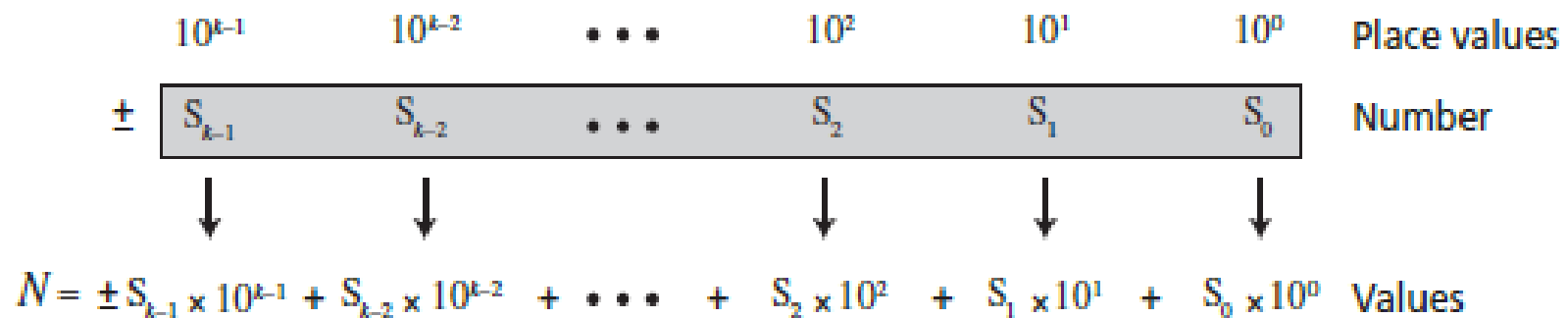
in which **S** is the set of **symbols**, **b** is the **base** (or **radix**).

2.2.1 The decimal system (base 10)

- The word decimal is derived from the Latin root **decem** (ten).
- In this system the **base $b = 10$** and we use ten symbols **$S = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}$**
- The symbols in this system are often referred to as **decimal digits** or just **digits**.

Integers $N = \pm s_{k-1} \times 10^{k-1} + s_{k-2} \times 10^{k-2} + \dots + s_2 \times 10^2 + s_1 \times 10^1 + s_0 \times 10^0$

Figure 2.1 Place value for an integer in decimal system



Example 2.1 The following shows the place values for the integer +224 in the decimal system.

		10^2	10^1	10^0	Place values
		2	2	4	Number
N	$=$	$+$	$+$	$+$	Values
		2×10^2	2×10^1	4×10^0	

- Note that the digit 2 in position 1 has the value 20, but the same digit in position 2 has the value 200.
- Also note that **we normally drop the plus sign, but it is implicit.**

Example 2.2 The following shows the place values for the decimal number -7508

		1000	100	10	1	Place values
		7	5	0	8	Number
N	$=$	$-$	$+$	$+$	$+$	Values
		$(7 \times 1000$	5×100	0×10	$8 \times 1)$	

Reals

- A real (a number with a fractional part) in the decimal system is also familiar.
- For example, we use this system to show dollars and cents (\$23.40).
- We can represent a real as

Integral part

Fractional part

$$R = \pm s_{k-1} \times 10^{k-1} + \dots + s_1 \times 10^1 + s_0 \times 10^0 + s_{-1} \times 10^{-1} + \dots + s_{-l} \times 10^{-l}$$

Example 2.7 The following shows the place values for the real number **+24.13**

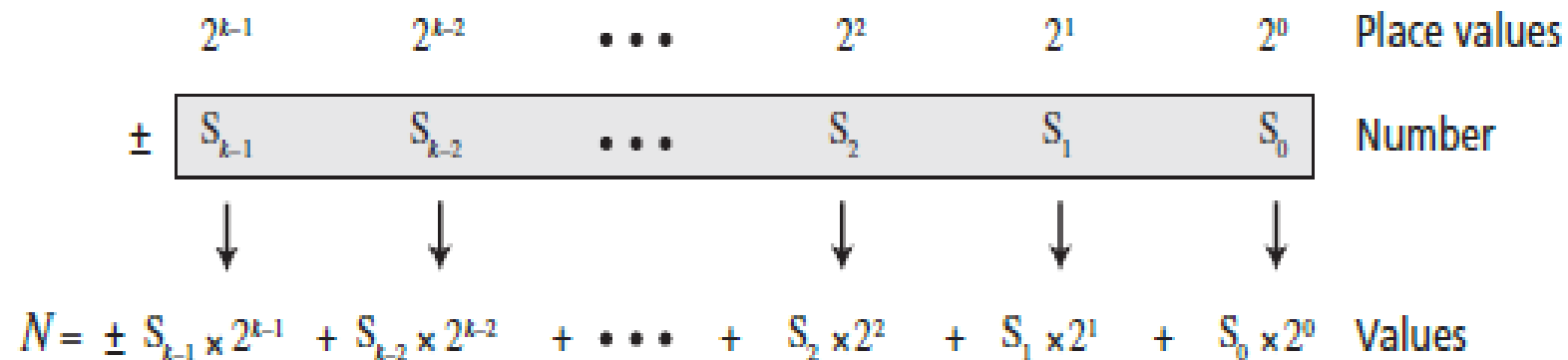
	10^1	10^0		10^{-1}	10^{-2}	Place values
	2	4	•	1	3	Number
$R = +$	2×10	$+ 4 \times 1$	$+$	1×0.1	$+ 3 \times 0.01$	Values

2.2.2 The binary system (base 2)

- The word binary is derived from the Latin root **bini** (or two by two).
- In this system the **base b = 2** and we use only two symbols **S = { 0, 1 }**.
- The symbols in this system are often referred to as **binary digits** or **bits** (binary digit).

Integers $N = \pm s_{k-1} \times 2^{k-1} + s_{k-2} \times 2^{k-2} + \dots + s_2 \times 2^2 + s_1 \times 2^1 + s_0 \times 2^0$

Figure 2.2 *Place values in an integer in the binary system*



Example 2.11 The following shows that the number **(11001)₂** in binary is the same as **25** in decimal. The subscript 2 shows that the base is 2.

	16		8		4		2		1	
	2^4		2^3		2^2		2^1		2^0	Place values
	1		1		0		0		1	Number
$N =$	1×2^4	+	1×2^3	+	0×2^2	+	0×2^1	+	1×2^0	Decimal

The equivalent decimal number is $N = 16 + 8 + 0 + 0 + 1 = 25$

Reals

A real—a number with an optional fractional part—in the binary system can be made of k bits on the left and l bits on the right:

$$R = \pm \underbrace{S_{k-1} \times 2^{k-1} + \dots + S_1 \times 2^1 + S_0 \times 2^0}_{\text{Integral part}} \cdot \underbrace{S_{-1} \times 2^{-1} + \dots + S_{-l} \times 2^{-l}}_{\text{Fractional part}}$$

Example 2.13 The following shows that the number $(101.11)_2$ in binary is equal to the number **5.75** in decimal.

	4	2	1	0.5	0.25	
	2^2	2^1	2^0	2^{-1}	2^{-2}	Place values
	1	0	1	1	1	Number
R =	1×2^2	0×2^1	1×2^0	1×2^{-1}	1×2^{-2}	Values
	+	+	+	+	+	
R =	4	0	1	0.5	0.25	= 5.75

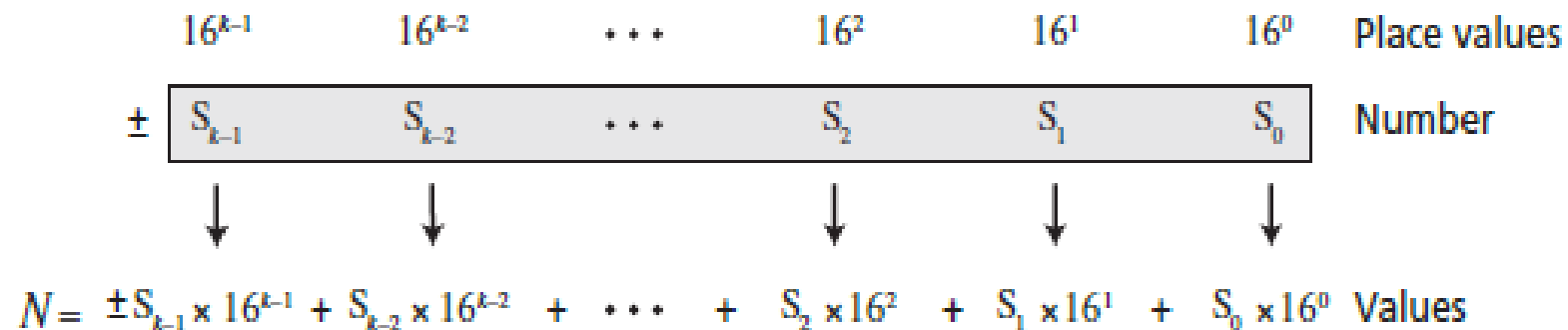
The hexadecimal system (base 16)

- The word **hexadecimal** is derived from the Greek root **hex** (six) and the Latin root **decem** (ten).
- In this system the **base b = 16** and we use sixteen symbols to represent a number. The set of symbols is **S = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F }**

10 11 12 13 14 15

Integers $N = \pm s_{k-1} \times 16^{k-1} + s_{k-2} \times 16^{k-2} + \dots + s_2 \times 16^2 + s_1 \times 16^1 + s_0 \times 16^0$

Figure 2.3 Place values in an integer in the hexadecimal system



Example 2.16 The following shows that the number $(2AE)_{16}$ in hexadecimal is equivalent to **686** in decimal.

	256		16		1	
	16^2		16^1		16^0	Place values
	2		A		E	Number
N =	2×16^2	+	10×16^1	+	14×16^0	Values

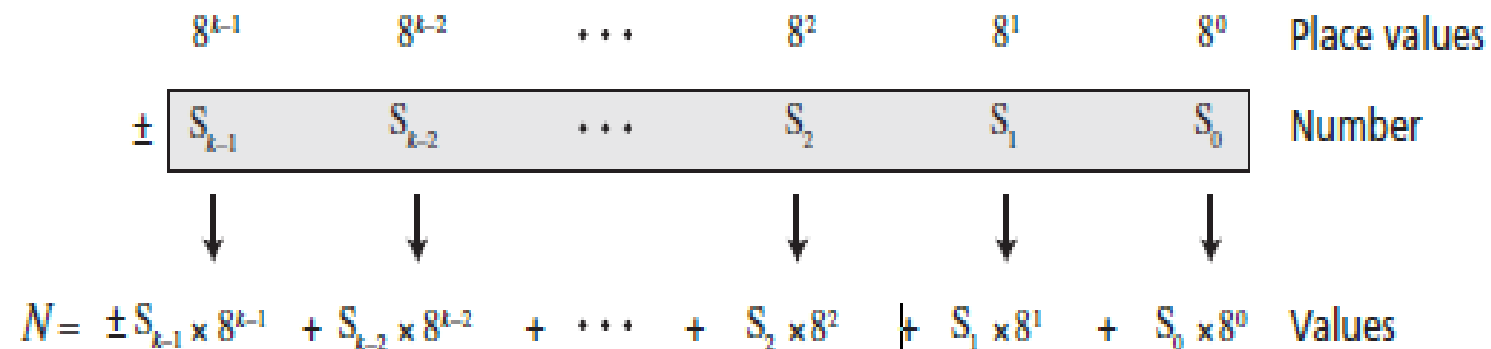
The equivalent decimal number is $N = 512 + 160 + 14 = 686$

2.2.4 The octal system (base 8)

- The word octal is derived from the Latin root **octo** (eight).
- In this system the **base b = 8** and we use eight symbols to represent a number.
- The set of symbols is **S = { 0, 1, 2, 3, 4, 5, 6, 7 }**

Integers $N = \pm s_{k-1} \times 8^{k-1} + s_{k-2} \times 8^{k-2} + \dots + s_2 \times 8^2 + s_1 \times 8^1 + s_0 \times 8^0$

Figure 2.4 Place values in an integer in the octal system



Example 2.18 The following shows that the number **(1256)₈** in octal is the same as **686** in decimal.

	512	64	8	1	
	8^3	8^2	8^1	8^0	Place values
	1	2	5	6	Number
N =	1×8^3	$+$ 2×8^2	$+$ 5×8^1	$+$ 6×8^0	Values

Note that the decimal number is $N = 512 + 128 + 40 + 6 = 686$

2.2.5 Summary of the four positional systems

Table 2.1 Summary of the four positional number systems

System	Base	Symbols	Examples
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	2345.56
Binary	2	0, 1	(1001.11) ₂
Octal	8	0, 1, 2, 3, 4, 5, 6, 7	(156.23) ₈
Hexadecimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F	(A2C.A1) ₁₆

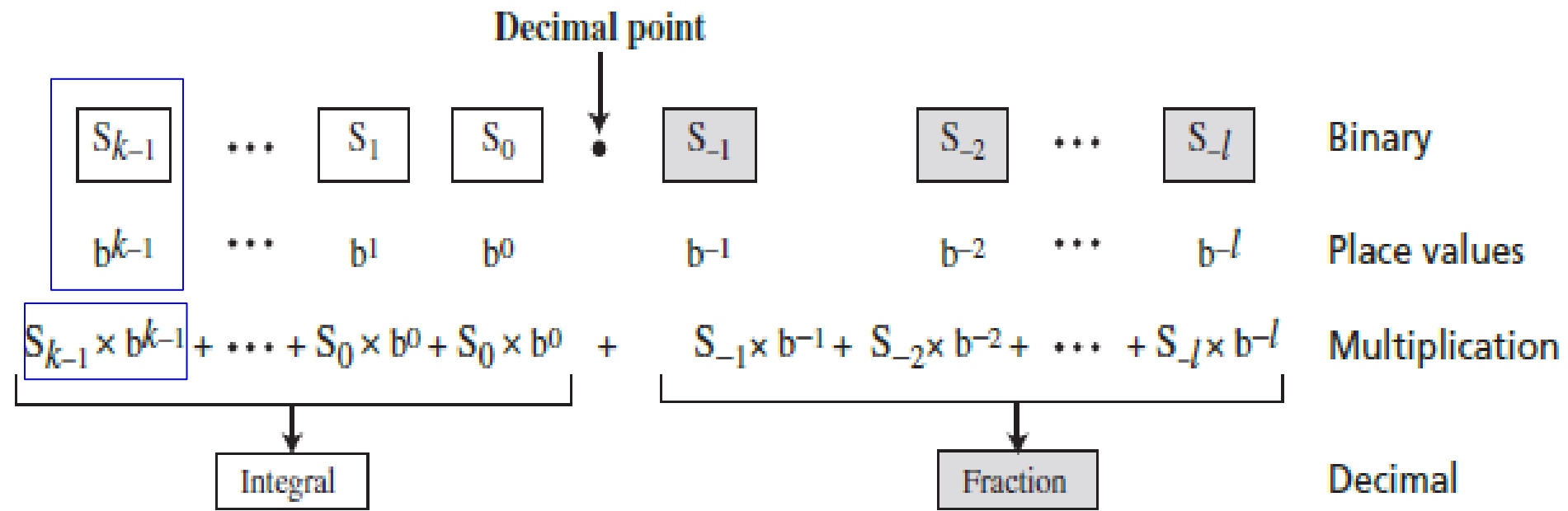
Table 2.2 Comparison of numbers in the four systems

Decimal	Binary	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

2.2.6 Conversion

From base-b to decimal conversion

Figure 2.5 Converting other bases to decimal



Example 2.19 The following shows how to convert the binary number $(110.11)_2$ to decimal: **$(110.11)_2 = 6.75$**

Binary	1	1	0	•	1	1
Place values	2^2	2^1	2^0		2^{-1}	2^{-2}
Partial results	4	2	0	+	0.5	0.25
Decimal: 6.75						

Example 2.20 The following shows how to convert the hexadecimal number $(1A.23)_{16}$ to decimal.

Hexadecimal	1	A	•	2	3
Place values	16^1	16^0		16^{-1}	16^{-2}
Partial result	16	10	+	0.125	0.012
Decimal: 26.137					

$(1A.23)_{16} \approx 26.137$

- Note that the result in the decimal notation is not exact, because $3 \times 16^{-2} = 0.01171875$. We have rounded this value to three digits (0.012)

Example 2.21 The following shows how to convert $(23.17)_8$ to decimal.

Octal	2	3	•	1	7
Place values	8^1	8^0		8^{-1}	8^{-2}
Partial result	16	3	+	0.125	0.109
Decimal: 19.234					$1/256 = 0.00390625$

$(23.17)_8 \approx 19.234$

- We have rounded up $7 \times 8^{-2} = 0.109375$

Decimal to any base

Figure 2.6 Converting the integral part of a decimal number to other systems

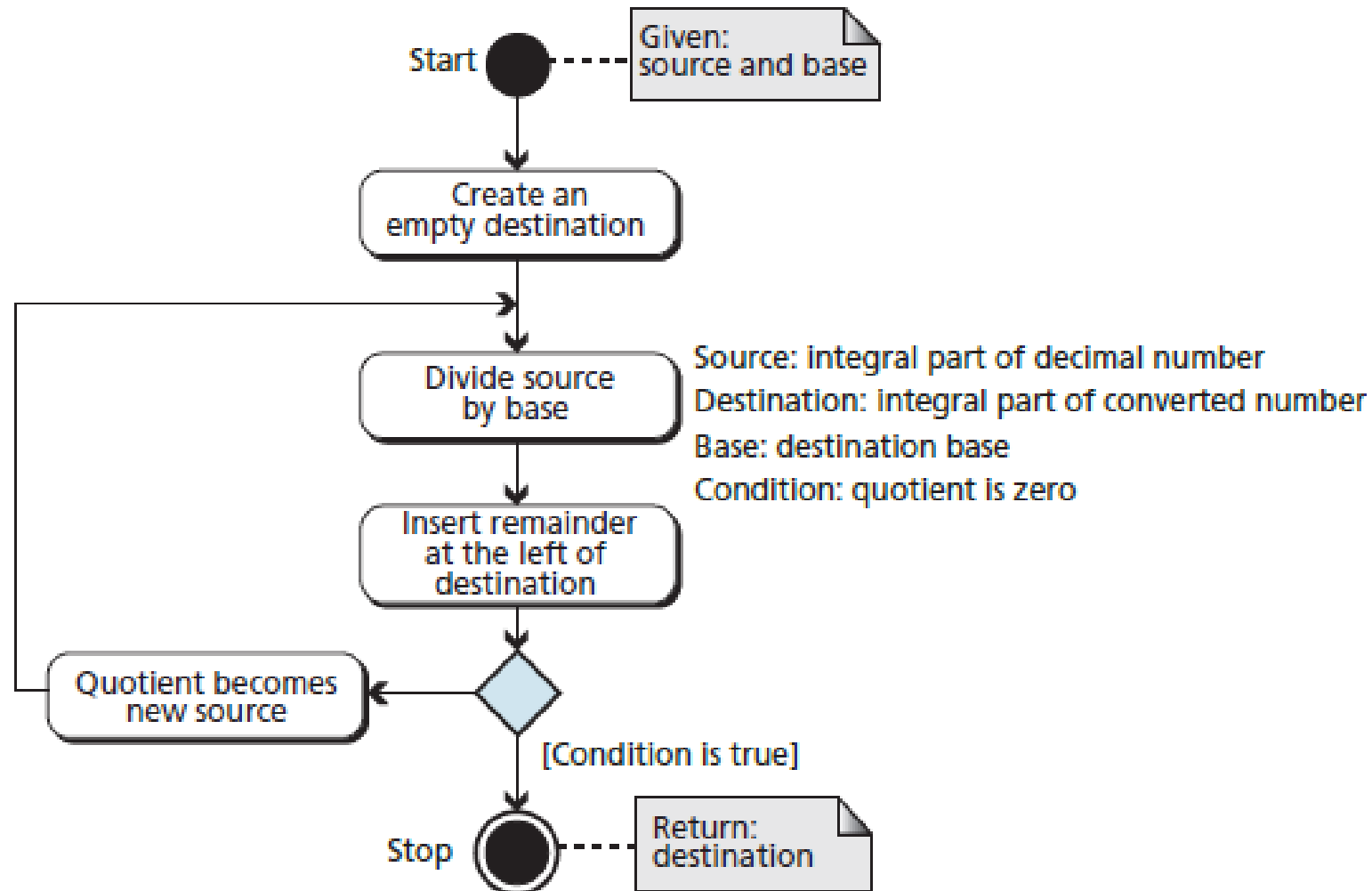
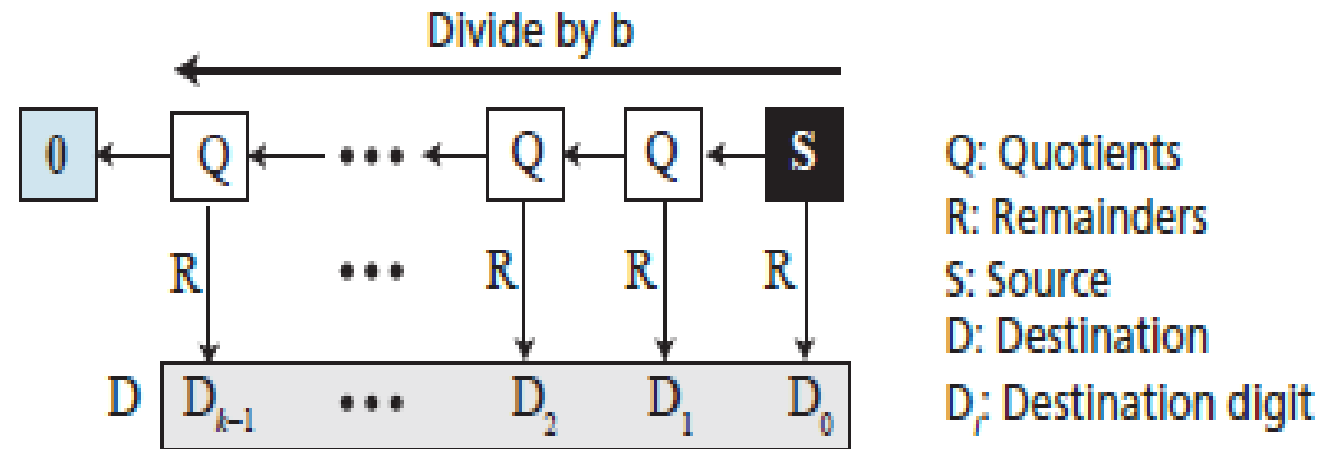
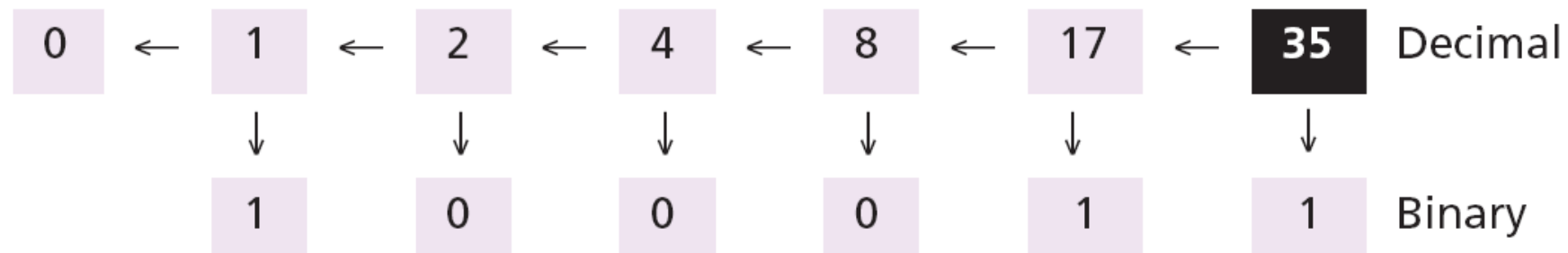


Figure 2.7 Converting the integral part

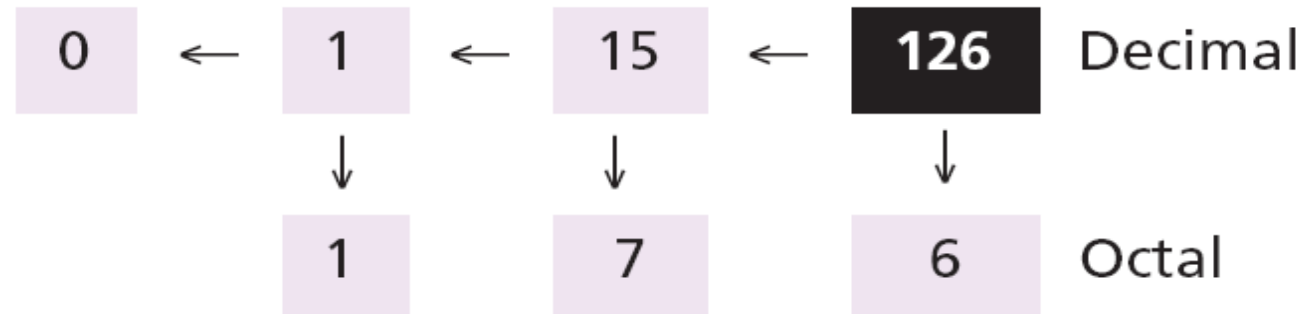


Example 2.22 $35 = (100011)_2$



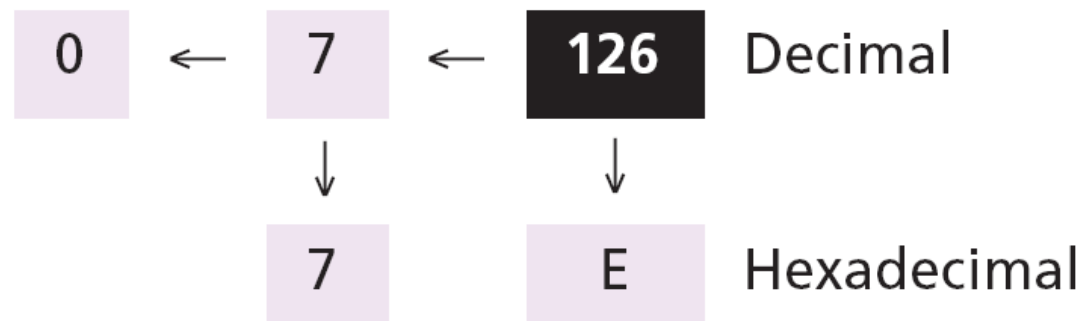
Example 2.23 The following shows how to convert 126 in decimal to its equivalent in the octal system. We move to the right while continuously finding the quotients and the remainder of division by 8.

The result is $126 = (176)_8$



Example 2.24 The following shows how we convert 126 in decimal to its equivalent in the hexadecimal system. We move to the right while continuously finding the quotients and the remainder of division by 16.

The result is $126 = (7E)_{16}$



Converting the fraction part

Figure 2.8 Algorithm to convert the fractional part of a decimal number to other bases

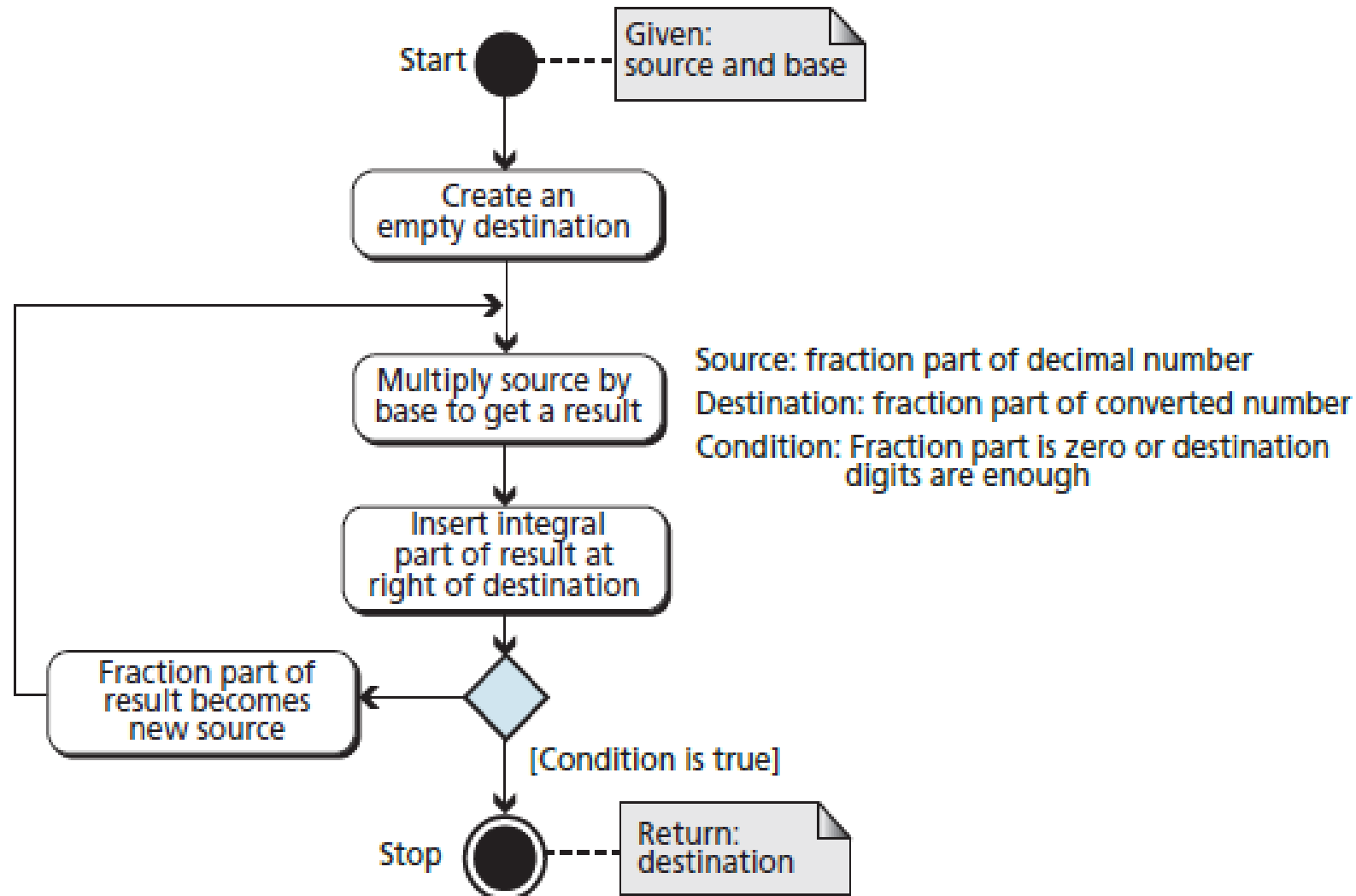
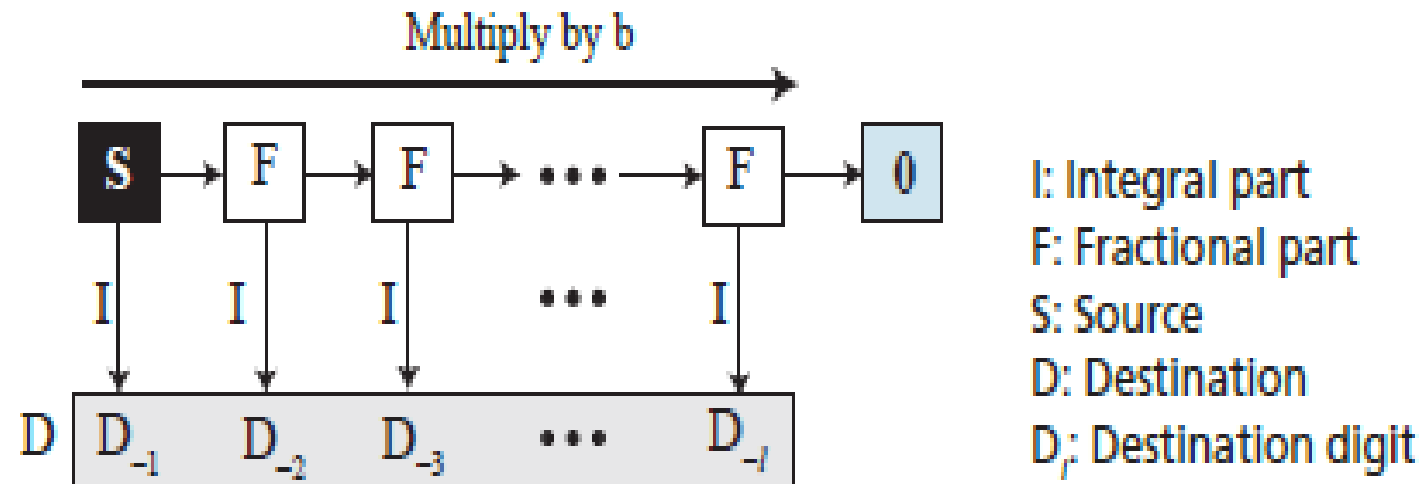


Figure 2.9 Converting the fractional part



Note:

The fraction may never become zero.

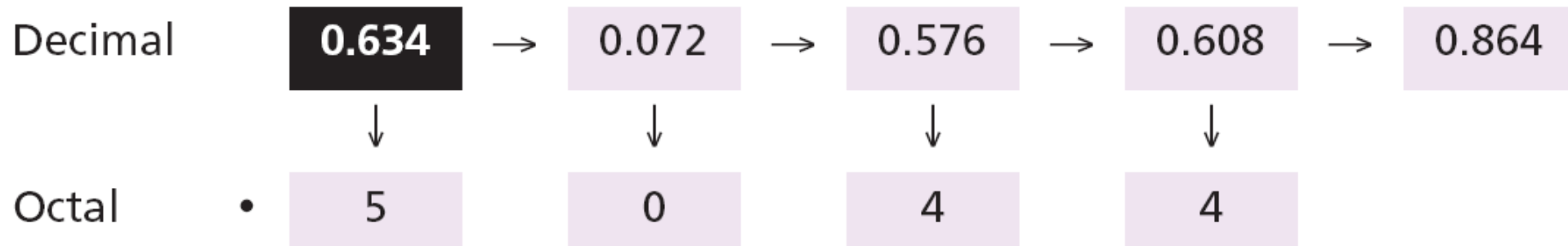
Stop when enough digits have been created.

Example 2.25 Convert the decimal number 0.625 to binary.

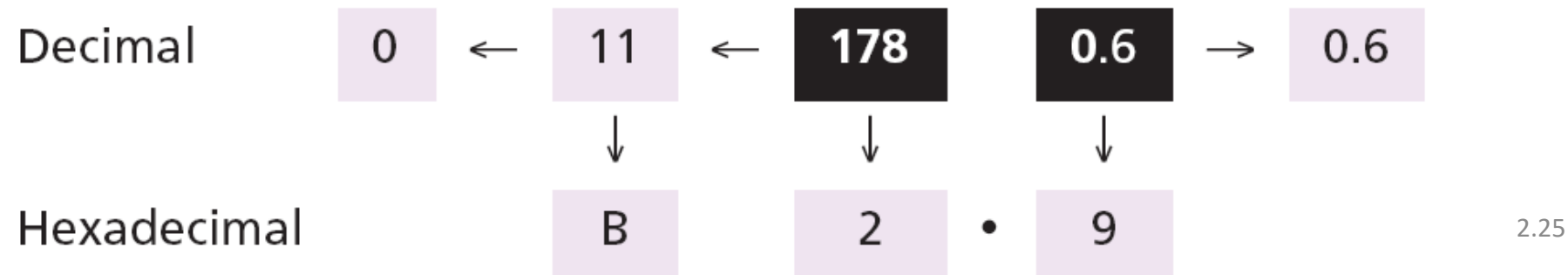
$$0.625 = (0.101)_2$$



Example 2.26 The following shows how to convert 0.634 to octal using a maximum of four digits. $0.634 = (0.5044)_8$



Example 2.27 $178.6 = (B2.9)_{16}$



Example 2.28 An alternative method for converting a small decimal integer (usually less than 256) to binary is to break the number as the sum of numbers that are equivalent to the binary place values shows: $165 = (10100101)_2$

Place Values	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
Decimal equivalent	128	64	32	16	8	4	2	1
Decimal 165 =	128		32			4		1
Binary	1	0	1	0	0	1	0	1

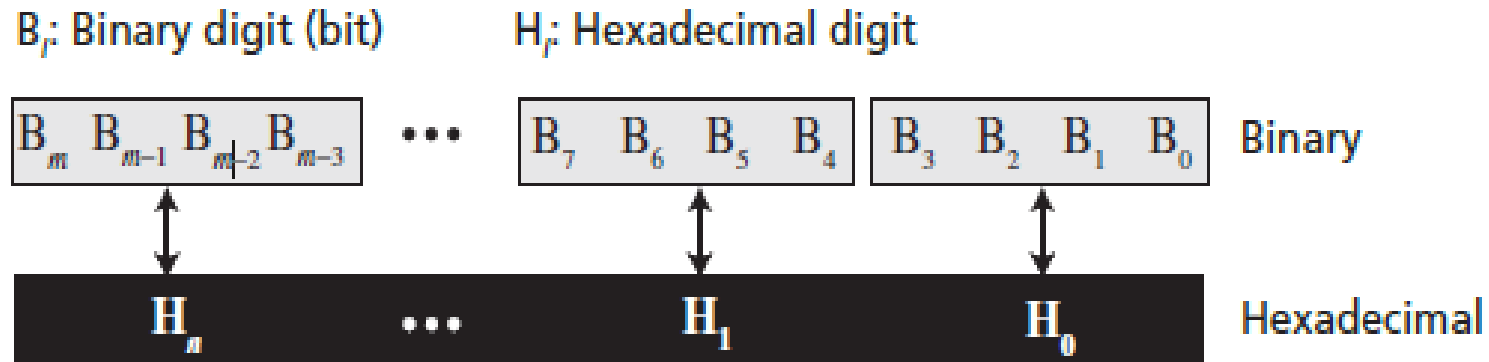
Example 2.29 A similar method can be used to convert a decimal fraction to binary when the denominator is a power of two:

27/64 = (0.011011)₂

Place Values	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}
Decimal equivalent	1/2	1/4	1/8	1/16	1/32	1/64	1/128
Decimal 27/62 =		16/64	8/64		2/64	1/64	
Binary	0	1	1	0	1	1	

Binary-hexadecimal conversion

Figure 2.10 *Binary to hexadecimal and hexadecimal to binary*

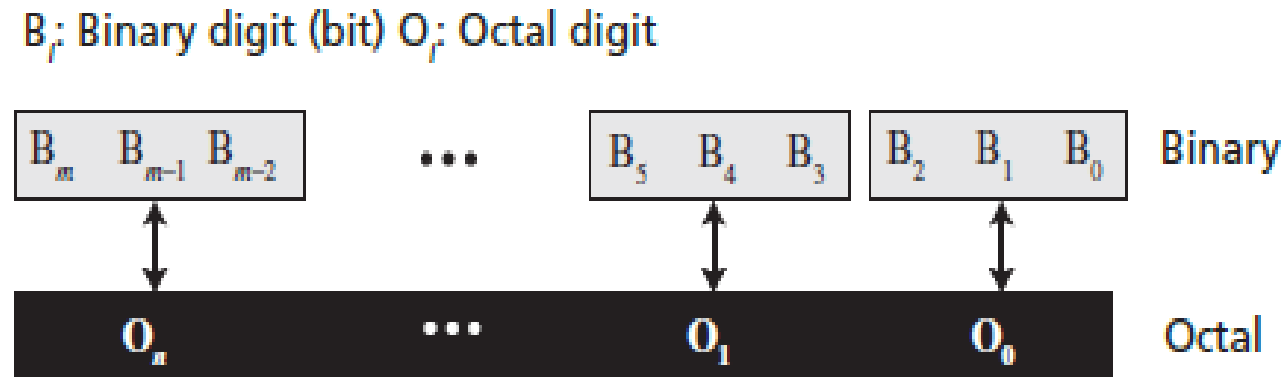


Example 2.30 $(10011100010)_2 = (100 \ 1110 \ 0010)_2 = (4E2)_{16}$

Example 2.31 $(24C)_{16} = (0010 \ 0100 \ 1100)_2 = (1001001100)_2$

Binary-octal conversion

Figure 2.11 *Binary to hexadecimal conversion*

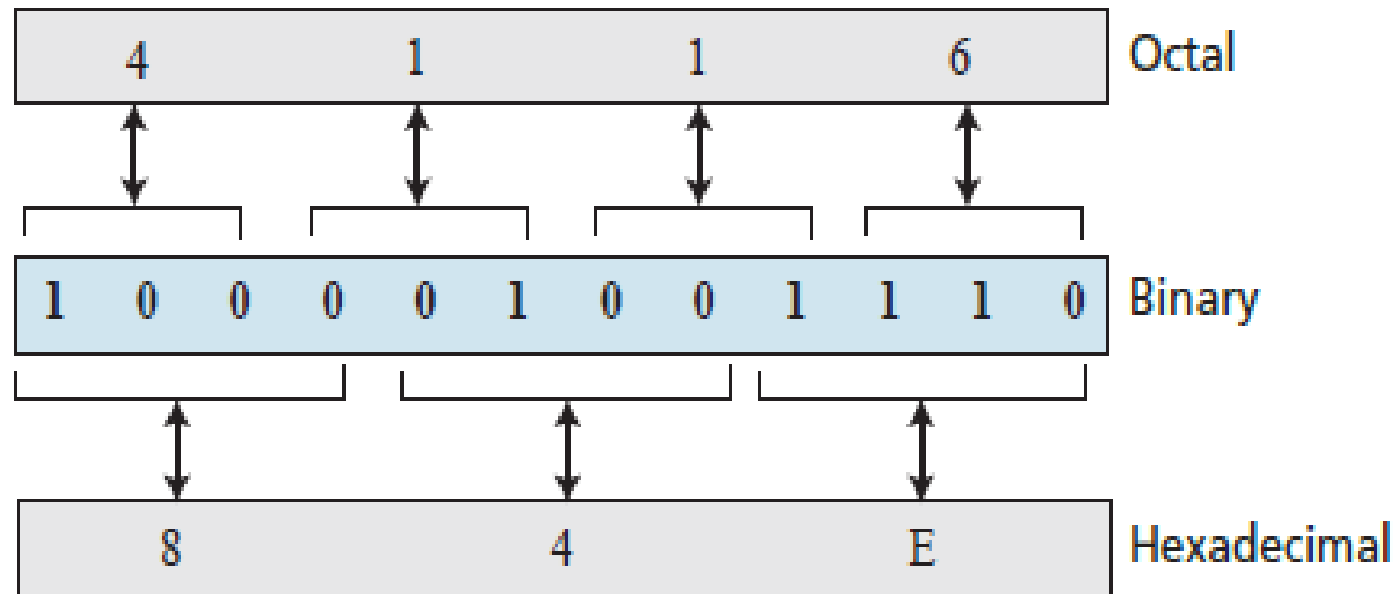


Example 2.32 $(101110010)_2 = (101 \ 110 \ 010)_2 = (562)_8$

Example 2.33 $(24)_8 = (010 \ 100)_2 = (10100)_2$

Octal-hexadecimal conversion

Figure 2.12 *Octal to hexadecimal and hexadecimal to octal conversion*



Example 2.34 Find the minimum number of binary digits required to store decimal integers with a maximum of 6 digits.

Solution: $k = 6$, $b_1 = 10$, and $b_2 = 2$. Then

$$x = \lceil k \times (\log b_1 / \log b_2) \rceil = \lceil 6 \times (1 / 0.30103) \rceil = 20$$

- The largest six-digit decimal number is 999,999 and the largest 20-bit binary number is 1,048,575.
- Note that the largest number that can be represented by a 19-bit number is 524,287, which is smaller than 999,999. We definitely need twenty bits.

2.3 NONPOSITIONAL NUMBER SYSTEMS

- Although non-positional number systems are not used in computers, we give a short review here for comparison with positional number systems.
- A non-positional number system still uses a limited number of symbols in which each symbol has a value. However, the position a symbol occupies in the number normally bears no relation to its value—the value of each symbol is fixed.
- To find the value of a number, we add the value of all symbols present in the representation.
- In this system, a number is represented as: $s_{k-1} \dots s_2 s_1 s_0 \bullet s_{-1} s_{-2} \dots s_{-l}$ and has the value of:
$$n = \pm \underbrace{s_{k-1} + \dots + s_1 + s_0}_{\text{(Integral part)}} + \underbrace{s_{-1} + s_{-2} + \dots + s_{-l}}_{\text{(Fractional part)}}$$

Roman numerals

- Roman numerals are a good example of a non-positional number system. This number system has a set of symbols $S = \{ I, V, X, L, C, D, M \}$.
- The values of each symbol is shown in Table 2.3

Table 2.3 Values of symbols in the Roman number system

<i>Symbol</i>	<i>I</i>	<i>V</i>	<i>X</i>	<i>L</i>	<i>C</i>	<i>D</i>	<i>M</i>
Value	1	5	10	50	100	500	1000

To find the value of a number, we need to add the value of symbols subject to specific rules.

- When a symbol with a smaller value is placed after a symbol having an equal or larger value, the values are added. VI $\rightarrow$ 5 + 1
- When a symbol with a smaller value is placed before a symbol having a larger value, the smaller value is subtracted from the larger one. IV $\rightarrow$ 5 - 1
- A symbol S_1 cannot come before another S_2 if $S_1 \times 10 < S_2$. For example, I or V cannot come before C. IC~~x~~ VC~~x~~ 小符號只能放在「下一個兩個等級」的大符號前面每種組合只能出現一次
- For larger numbers a bar is placed above any of the six symbols (all symbols except I) to express multiplication by 1000. For example, $\overline{V}$ = 5 000 and $\overline{M}$ = 1 000 000.
- Although Romans used the word *nulla* (nothing) to convey concept of zero, the Roman numerals lack a zero digit in their system.

The following shows some Roman numbers and their values.

III	→	$1 + 1 + 1$	=	3
IV	→	$5 - 1$	=	4
VIII	→	$5 + 1 + 1 + 1$	=	8
XVIII	→	$10 + 5 + 1 + 1 + 1$	=	18
XIX	→	$10 + (10 - 1)$	=	19
LXXII	→	$50 + 10 + 10 + 1 + 1$	=	72
CI	→	$100 + 1$	=	101
MMVII	→	$1000 + 1000 + 5 + 1 + 1$	=	2007
MDC	→	$1000 + 500 + 100$	=	1600

Unary numeral system/unary code 一進位數字

Decimal	Unary number	Unary code
0	ϵ - empty string	0
1	1	10
2	11	110
3	111	1110
4	1111	11110
5	11111	111110
6	111111	1111110
7	1111111	11111110
8	11111111	111111110
9	111111111	1111111110
10	1111111111	11111111110

C/C++ Numbers <https://www.onlinegdb.com/>

```
#include <iostream>
using namespace std;
int main()
{
    cout << "Decimal " << 65 << endl;
    cout << "Hexadecimal 0x65 = " << 0x65 << " in decimal" << endl;
    cout << "Octal 065 = " << 065 << " in decimal" << "\n\n";

    cout << "Decimal value " << 65 << endl;
    cout << "Represent 65 in hexadecimal = " << hex << 65 << endl;
    cout << "Represent 65 in octal = " << oct << 65 << "\n\n";

    cout << showbase;
    cout << "Decimal value " << dec << 65 << endl;
    cout << "Represent 65 in hexadecimal = " << hex << 65 << endl;
    cout << "Represent 65 in octal = " << oct << 65 << "\n\n";

    cout << noshowbase;
    cout << "Represent 65 in hexadecimal = " << hex << 65 << endl;

    return 0;
}
```

output

Decimal 65
Hexadecimal 0x65 = 101 in decimal
Octal 065 = 53 in decimal

Decimal value 65
Represent 65 in hexadecimal = 41
Represent 65 in octal = 101

Decimal value 65
Represent 65 in hexadecimal = 0x41
Represent 65 in octal = 0101

Represent 65 in hexadecimal = 41

...Program finished with exit code 0
Press ENTER to exit console.

Exercises

- P. 38 2.5.4 Problems
 - P2-1, P2-2, P2-6, P2-8, P2-10, P2-12, P2-13, P2-22, P2-23, P2-24